SAVOR – MODERN DINING EXPERIENCE

A RESPONSIVE RESTAURANT WEBSITE USING HTML, CSS AND JAVASCRIPT

CS23531 - MINI-PROJECT REPORT

Submitted by

SABTHA SHREE J M(241001208)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY



DEPARTMENT OF INFORMATION TECHNOLOGY
RAJALAKSHMI ENGINEERING COLLEGE

JULY 2026

BONAFIDE CERTIFICATE

Certified that this project report titled SAVOR – MODERN DINING EXPERIENCE: A RESPONSIVE RESTAURANT WEBSITE USING HTML, CSS AND JAVASCRIPT is the bonafide work of SABTHA SHREE J M(241001208) who carried out the work during her Mini-Project work. Certified further that based on the task given by the training organization, she has completed this project under their Supervision.

SIGNATURE OF THE REVIEWER

SIGNATURE OF THE HOD

Dr. P. Neelaveni
Head of the Department
Department of Information Technology
Rajalakshmi Engineering College

SIGNATURE OF THE CLASS IN-CHARGE

Submitted to Project Viva Voce Examination held on _____

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our chairman **Mr. S. Meganathan**, B.E., F.I.E., for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan**, for her enthusiastic motivation which inspired us a lot in completing this project and Vice-Chairman **Mr. Abhay Shankar Meganathan**, B.E., M.S., for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college principal, **Dr. S. N. Murugesan**, M.E., Ph.D., for his kind support and facilities to complete our work on time. We extend heartfelt gratitude to **Dr. P. Neelaveni**, Professor and Head of the Department of Information Technology, for her guidance and encouragement throughout the work.

We extend our thanks to our project guide, faculty members, parents, and friends for their direct and indirect involvement in the successful completion of this project, for their encouragement and support.

SABTHA SHREE J M

TABLE OF CONTENTS

S.NO	CONTENTS	PAGE NO
—	Abstract	5
1	Introduction	6
2	Literature Review and Background Study	6–7
3	System Requirements	8
4	System Design and Methodology	9–11
5	Technology Stack	12
6	Implementation	13–17
7	Results and Discussion	18–22
8	Challenges and Issues Faced	23
9	Testing	24–25
—	Appendix A — Menu Data Model	26
—	Appendix B — Reservation Form Markup	26
—	Appendix C — Design Tokens and Responsive Rules	27
—	Appendix D — Interaction Logic	28–30
10	Conclusion and Future Scope	31
11	References	32
12	System Usage / Use Case Description	32
13	User Manual — Running the Application	33–34
14	Glossary of Terms	35
—	SDG Mapping	36

Abstract

SAVOR is a responsive, single-page restaurant website that presents a modern fine-dining brand, a browsable seasonal Indian menu and an online table-reservation form. The project demonstrates how a complete, polished and accessible user experience can be delivered with only the core web technologies — HTML5, CSS3 and vanilla JavaScript — with no framework, no build step, no backend and no database. The site is organised into six sections (Home, Menu, About Us, Reservations, Contact and Footer) and works on desktop, tablet and mobile screens.

The application was developed in three stages. In the first stage, the information architecture and visual identity were defined: a cream, dark-brown, black and gold palette expressed as CSS custom properties, paired with Cormorant Garamond and Hanken Grotesk typefaces. In the second stage, the interactive behaviour was implemented in JavaScript: a data-driven menu of 24 dishes rendered from a single MENU array, category and vegetarian/non-vegetarian filters, an accessible dish-details dialog, an “Add Dish” form with validation, a mobile navigation drawer with scroll-spy, and toast notifications. In the third stage, a reservation module was added with field-level validation (name, e-mail, phone, party size, date and time), a 90-day booking window, same-day time checks and a confirmation view. Each dish is shown with a bundled photograph from the project’s images folder, and a small module (art.js) can draw an SVG illustration as a fallback, so that the menu never shows an empty picture.

The finished site (version 2, with photographic menu images) was verified with 20 automated browser checks in Chromium, all of which behaved as expected, and the report also records a short list of known limitations — such as reservations not being transmitted to a server and added dishes not persisting across reloads — together with recommended improvements.

1. Introduction

A restaurant's website is often the first impression a guest has of the business. Guests want to see what is on the menu, check whether a dish suits their diet, understand the atmosphere of the place and book a table, ideally within a few taps on a phone. Printed menus and telephone bookings cannot offer this convenience at any hour, and generic third-party listing pages give the restaurant little control over how it is presented.

SAVOR – Modern Dining Experience addresses this need with a purpose-built front-end website for a fictional fine-dining restaurant in Chennai that serves seasonal Indian cuisine cooked over open fire. The site combines brand storytelling (hero, About Us and Chef's favourites) with practical tools (a filterable menu, dish details, a reservation form and opening-hour and contact information).

The project was built as a student project for Web Programming and deliberately restricts itself to the fundamental web platform. All content is rendered in the browser from a JavaScript data array, layout is handled with CSS Grid and Flexbox, and interactivity is written in plain JavaScript using modern browser features such as the native <dialog> element, IntersectionObserver and CSS custom properties. This keeps the project lightweight, easy to understand, easy to deploy (it runs by double-clicking index.html) and a good demonstration of web fundamentals.

Particular attention was paid to quality attributes that are often neglected in student websites: accessibility (a skip link, keyboard-operable controls, ARIA attributes, visible focus outlines and reduced-motion support), graceful degradation (a no-script message and an offline fallback for images) and safe handling of user input (all user-supplied text is escaped before it is inserted into the page). This report documents the background, requirements, design, implementation, results, testing, challenges and future scope of the system.

2. Literature Review and Background Study

2.1 The Role of Web Presence in Hospitality

For restaurants and cafés, an online presence is now a basic business requirement. Guests commonly search for a venue, read the menu and decide where to eat on a mobile phone before they ever visit. A dedicated website lets a restaurant control its message, show its full menu with prices and dietary information, and offer direct booking without paying commission to aggregator platforms. Clear menu presentation, visible opening hours, an obvious call to action (such as "Book a Table") and fast load times are consistently recommended as the core elements of an effective restaurant website.

2.2 Responsive Web Design

Responsive web design, introduced by Ethan Marcotte in 2010, combines fluid grids, flexible images and CSS media queries so that one code base adapts to any screen size. Modern CSS Grid and Flexbox make these layouts simpler and more robust than the float-based layouts used earlier. SAVOR follows this approach: the page uses a fluid container, grid-based card layouts that collapse from four columns to two and then one, and three breakpoints (992 px, 860 px and 640 px) that also switch the navigation from a horizontal bar to a full-screen mobile drawer.

2.3 Client-Side Rendering with the DOM

The Document Object Model (DOM) represents an HTML page as a tree of objects that JavaScript can read and modify. Rendering content from a data structure — rather than hand-writing repeated HTML — makes pages easier to maintain, because a change to the data is reflected everywhere it is used. In SAVOR, one MENU array of 24 objects drives the Featured section, the Menu grid, the dish-details dialog and the filters. Event delegation, where a single listener on the document handles clicks for many dynamically created buttons, keeps the code short and avoids re-attaching listeners after each re-render.

2.4 Web Accessibility

The Web Content Accessibility Guidelines (WCAG) 2.2, published by the W3C, define how to make web content perceivable, operable, understandable and robust. Practical techniques include semantic HTML landmarks, a “skip to content” link, labelled form controls, error messages that are announced to assistive technology, sufficient colour contrast, visible keyboard focus and respect for the user’s reduced-motion preference. The WAI-ARIA Authoring Practices Guide further describes patterns for dialogs, toggle buttons and live regions. SAVOR applies these techniques throughout (see Section 6.7).

2.5 Form Validation and User Experience

Good form design validates input at the right moment and explains problems in plain language. Research and design guidance generally recommend validating a field when the user leaves it, clearing the error as soon as it is fixed, placing the message next to the field, and focusing the first invalid field on submit. The browser’s built-in validation can be supplemented with custom rules for domain-specific checks, such as rejecting reservation dates in the past or beyond a booking window. SAVOR implements this pattern for both the reservation form and the Add Dish form.

2.6 Summary

The background study shows that (i) a restaurant website must combine brand presentation with practical tools such as a menu and booking, (ii) a responsive, data-driven, standards-based front end can deliver this without heavy frameworks, and (iii) accessibility and careful

validation are essential parts of the user experience rather than optional extras. These three conclusions directly shaped the design decisions described in the following chapters.

3. System Requirements

3.1 Hardware Requirements

- Any desktop, laptop, tablet or smartphone capable of running a current web browser; no GPU or special hardware is needed
- Approximately 4 GB RAM and a standard display for comfortable development and testing
- Internet connectivity is optional: it is required only to load Google Fonts and the Unsplash photographs used in the hero and About sections

3.2 Software Requirements

- A current version of Google Chrome, Microsoft Edge or Mozilla Firefox (support for the <dialog> element, CSS custom properties, CSS Grid and IntersectionObserver is required)
- A text or code editor for development (for example, Visual Studio Code)
- Optional: a simple local web server (such as python -m http.server) for testing; the site also runs directly from the file system
- Playwright with Chromium — used in this report to run automated browser checks and capture screenshots

3.3 External Resources Used

- **Google Fonts** — Cormorant Garamond (headings) and Hanken Grotesk (body text), with Georgia and system fonts as fallbacks
- **Unsplash** — photographs for the hero banner and the About Us section only; if they fail to load, the page remains fully usable
- **images/ folder** — 24 dish photographs (JPEG, about 480 × 438 px, 1.1 MB in total) stored with the project, so the menu photographs load without an internet connection
- **art.js (own module)** — generates SVG illustrations as data URIs; used as a fallback for dishes that have no photograph, such as dishes added at run time
- **Inline SVG** — logo mark, icons, favicon and the stylised map placeholder

3.4 Functional Requirements

- Display a hero section with the restaurant name, tagline, key facts and calls to action
- Show three featured dishes and a full menu of dishes grouped as Starters, Main Course, Desserts and Beverages
- Allow guests to filter the menu by category and by vegetarian / non-vegetarian preference, and to combine both filters
- Show dish details (description, ingredients, allergen/serving note and price) in a dialog
- Allow a new dish to be added to the menu through a validated form

- Provide a reservation form with validation and a confirmation message
- Provide opening hours, address, phone, e-mail and a map placeholder

3.5 Non-Functional Requirements

- **Responsive** — usable from 320 px phone screens to large desktop monitors
- **Accessible** — keyboard operable, labelled controls, ARIA live regions, visible focus, reduced-motion support
- **Portable** — runs without installation or build tools; delivered both as separate files and as a single standalone HTML file
- **Secure by construction** — user-entered text is escaped or inserted with textContent, never as raw HTML

4. System Design and Methodology

4.1 Overview of the Development Approach

The website was developed iteratively in three stages, each building on the previous one: (1) page structure and visual identity, (2) data-driven menu and interactions, and (3) reservation and form validation, followed by polishing for accessibility and responsiveness. The code is organised into four files with clear responsibilities: index.html holds the semantic structure, css/style.css holds all presentation, js/art.js generates fallback dish artwork and js/script.js holds the application logic and menu data. The images/ folder holds the 24 dish photographs. A further file, savor-standalone.html, bundles the same site into a single file for easy sharing (it still needs the images folder beside it).

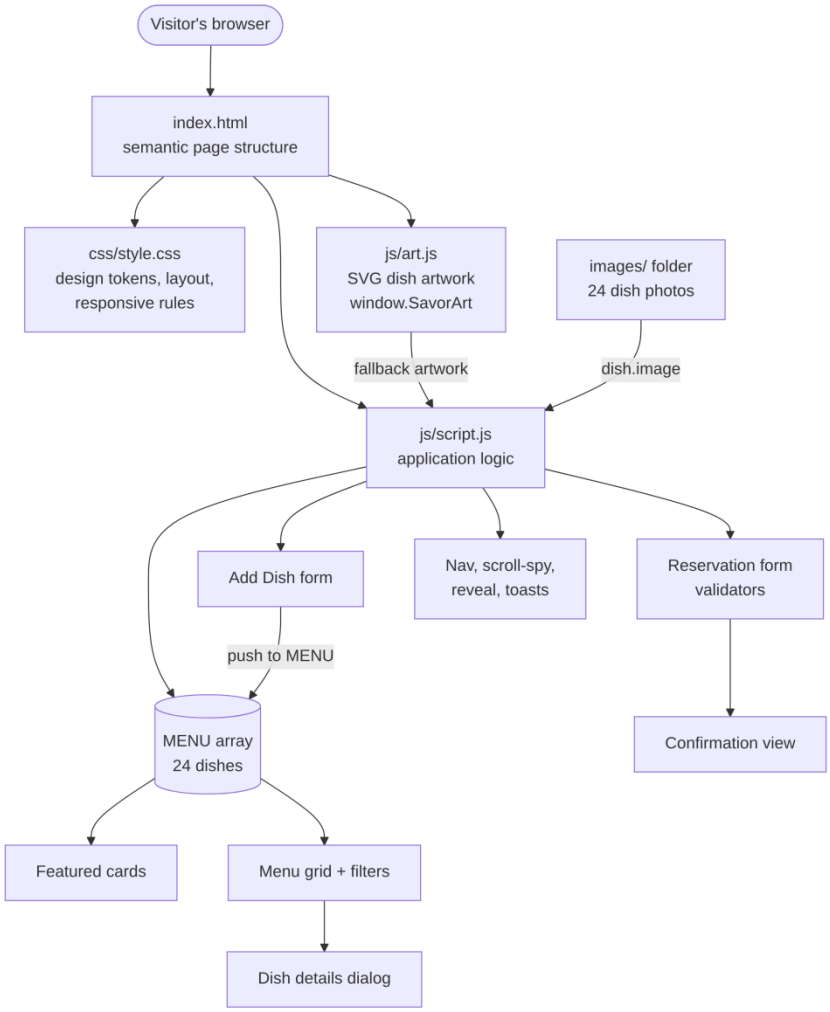


Fig 4.1 — Architecture of the SAVOR front end: files, data flow and interaction modules

4.2 Information Architecture

The site is a single page with anchor navigation. Each major section has an id that is referenced by the navigation bar, the footer and the call-to-action buttons, and a scroll-spy highlights the current section in the navigation.

Section	Anchor	Purpose and key content
Header / navigation	#siteHeader	Logo, five navigation links, “Book a Table” button; becomes solid on scroll; collapses to a hamburger drawer below 861 px
Hero	#home	Tagline “Good Food. Great Moments.”, introduction, Explore Menu and Book a Table buttons, and three key facts (hours, kitchen, seats)
Featured	#featured	Three “Chef’s favourites” cards generated from dishes flagged featured: true
Menu	#menu	Category filters, diet filters, dietary key, Add Dish button and the 24-dish grid
About Us	#about	Restaurant story, two photographs and three values (local first, made daily, warm service)
Reservations	#reservations	Booking notes and the reservation form with confirmation view

Section	Anchor	Purpose and key content
Contact	#contact	Opening-hours table, address, phone, e-mail and map placeholder
Footer	—	Brand blurb, social icons, quick links, address and copyright
Dialogs and toasts	#dishDialog, #addDishDialog, #toastRegion	Dish details, Add Dish form and transient notifications

4.3 Visual Design System

The visual identity is defined once, as CSS custom properties on :root, and reused everywhere. The palette is warm and low-key to suggest fine dining, with gold reserved for calls to action and accents. A darker gold (--gold-text) is provided for text on cream backgrounds so that contrast remains sufficient.

Token	Value	Use
--cream / --cream-deep	#f6efe0 / #ebdfc7	Page background and alternate section background
--paper	#fffaf0	Cards and form surfaces
--brown / --brown-soft	#2b1b12 / #4a3326	Primary text and dark surfaces
--black	#15110e	Header, mobile menu and footer
--gold / --gold-light	#b8924a / #d8b774	Buttons, accents, active states
--gold-text	#7d5f24	Gold text on light backgrounds (higher contrast)
--danger / --success	#a3341f / #3d6b45	Error messages and success states; non-veg and veg markers
--font-display	Cormorant Garamond	Headings (serif, with Georgia fallback)
--font-body	Hanken Grotesk	Body text (sans-serif, with system fallback)

4.4 Menu Data Model

Every dish is a plain JavaScript object in the MENU array at the top of script.js. Because all views are generated from this array, adding or editing a dish requires changing one place only.

Field	Type	Purpose
id	number	Unique identifier used by the View Details buttons (data-dish attribute)
category	string	One of starters, mains, desserts or beverages; used by the category filter
veg	boolean	true for vegetarian, false for non-vegetarian; drives the diet filter and the green/red marker
featured	boolean	When true, the dish appears in the Chef's favourites section

Field	Type	Purpose
price	number	Price in rupees; formatted with the en-IN locale (for example ₹1,200)
name, desc	string	Dish title and one-sentence description
ingredients, note	string	Shown in the details dialog; note holds allergen and serving information
image	string	Relative path of the dish photograph, for example images/06_paneer_tikka.jpg
artKey	string	Key into art.js that selects the fallback SVG illustration when no image is set

The shipped menu contains 24 dishes, six in each category (Starters, Main Course, Desserts and Beverages). Of these, 19 are vegetarian and 5 are non-vegetarian, and 3 are featured (Chicken Biryani, Paneer Butter Masala and Chocolate Lava Cake).

4.5 Application Flow

When the page loads, the script (wrapped in an immediately invoked function expression to avoid global variables) executes the following sequence:

- Adds the class js to the root element so that CSS can enable JavaScript-only effects such as scroll reveal
- Renders the featured cards and the full menu from the MENU array
- Attaches listeners for the category filters, diet filters, dish buttons, dialogs and the Add Dish form
- Populates the reservation form's guest (1–12) and time (12:00 pm–10:30 pm, every 30 minutes) drop-downs and sets the minimum and maximum selectable dates
- Attaches validators to each reservation field, the character counter and the submit handler
- Starts the scroll handler (sticky header, back-to-top button, scroll-spy) and the IntersectionObserver for reveal animations

From then on, user actions update shared state (the current category, the current diet filter and the MENU array) and re-render the affected part of the page.

5. Technology Stack

- **Markup:** HTML5 with semantic landmarks (header, nav, main, section, footer, dialog, address, dl)
- **Styling:** CSS3 with custom properties, Grid, Flexbox, transitions, animations and media queries (including prefers-reduced-motion)
- **Scripting:** Vanilla JavaScript (ES6+) — template literals, arrow functions, const/let, Array methods, Intl/toLocaleString, IntersectionObserver
- **Graphics:** Inline SVG and SVG data URIs generated by art.js; remote photographs from Unsplash
- **Typography:** Google Fonts — Cormorant Garamond and Hanken Grotesk

- **Data storage:** In-memory JavaScript array (no database, no browser storage)
- **Delivery:** Static files; separate-file version and a single-file standalone version
- **Verification:** Playwright (Chromium) for automated checks and screenshots
- **Development environment:** Any code editor and any modern browser with developer tools

6. Implementation

6.1 Page Structure and Styling

index.html defines the semantic structure and loads the fonts, style.css, art.js and script.js (in that order, because script.js depends on window.SavorArt). A skip link is the first focusable element, the logo and favicon are inline SVG, and the menu and featured containers include <noscript> messages. style.css begins with the design tokens (Section 4.3), a small reset and base typography, and then styles each component. Three media-query blocks (max-width 992 px, 860 px and 640 px) reorganise the grids, switch to the mobile navigation drawer and stack the forms; a fourth block honours prefers-reduced-motion by disabling animation and smooth scrolling.

6.2 Rendering the Menu

Each dish card is produced by a template function that escapes every piece of text before insertion. The image source is chosen with dish.image || artUri(dish.artKey), so the photograph is used when one is defined and the generated illustration otherwise. The card shows the picture, a veg/non-veg marker, the name, the price, a short description and a View Details button that carries the dish id in a data attribute.

```
function dishCardHTML(dish, index) {
  const uri = dish.image || artUri(dish.artKey);
  return `
    <article class="dish-card" style="animation-delay:${index * 50}ms"
       data-veg="${dish.veg}">
      <div class="img-wrap" data-label="${escapeHTML(dish.name)}">
        ${uri ? `<img src="${uri}" alt="${escapeHTML(dish.name)}">` : ""}
      </div>
      <div class="dish-body">
        <div class="dish-top">
          <h3>${dietDot(dish.veg)}<span>${escapeHTML(dish.name)}</span></h3>
          <span class="price">${formatPrice(dish.price)}</span>
        </div>
        <p class="dish-desc">${escapeHTML(dish.desc)}</p>
        <button type="button" class="link-btn" data-dish="${dish.id}"
           aria-label="View details for ${escapeHTML(dish.name)}">View Details</button>
      </div>
    </article>`;
}
```

The Featured section is rendered the same way from the dishes flagged featured, using larger overlay cards. If JavaScript is unavailable, the <noscript> notes explain that scripting is required to view the menu.

6.3 Category and Diet Filtering

Two independent state variables, currentCategory and currentDiet, are updated by the filter buttons. renderMenu() applies both in sequence, so any combination (for example, Main Course + Non-Veg Only) works, and displays a friendly message when nothing matches. The buttons use aria-pressed and a class to show which filter is active.

```
/* ---------- Filter state & render ---------- */
let currentCategory = "all";
let currentDiet = "all";

function renderMenu() {
  let items = currentCategory === "all"
    ? MENU
    : MENU.filter((d) => d.category === currentCategory);

  if (currentDiet === "veg")    items = items.filter((d) => d.veg);
  if (currentDiet === "nonveg") items = items.filter((d) => !d.veg);

  if (items.length === 0) {
    menuGrid.innerHTML = `<p class="menu-empty">No dishes match your
        selection.</p>`;
  } else {
    menuGrid.innerHTML = items.map(dishCardHTML).join("");
  }
}
```

6.4 Dish Details Dialog

Clicking any element with a data-dish attribute opens the native <dialog> via showModal(), which provides focus trapping, an inert background and Escape-to-close for free. The content is filled with textContent, and focus returns to the button that opened the dialog when it closes. Clicking the backdrop also closes it.

```
function openDish(id, trigger) {
  const dish = MENU.find((d) => d.id === Number(id));
  if (!dish) return;
  lastTrigger = trigger;

  const imgWrap = $("#dialogImg").parentElement;
  imgWrap.classList.remove("img-failed");
  imgWrap.dataset.label = dish.name;
  const uri = dish.image || artUri(dish.artKey);
  const dialogImg = $("#dialogImg");
  dialogImg.src = uri;
  dialogImg.alt = dish.name;
  $("#dialogCat").textContent = CATEGORY_LABELS[dish.category];
  $("#dialogTitle").textContent = dish.name;
  $("#dialogDiet").innerHTML = `${dietDot(dish.veg)} ${dish.veg ? "Vegetarian" :
      "Non-vegetarian"}`;
  $("#dialogDesc").textContent = dish.desc;
  $("#dialogIng").textContent = dish.ingredients;
  $("#dialogNote").textContent = dish.note;
  $("#dialogPrice").textContent = formatPrice(dish.price);

  if (typeof dialog.showModal === "function") dialog.showModal();
  else dialog.setAttribute("open", "");
}
```

6.5 Adding a Dish

The Add Dish button opens a second dialog containing a form for name, category, price, dietary type, description, and optional ingredients and notes. On submit the values are validated, a new object is pushed to MENU with a unique id and a fallback artwork key chosen by SavorArt.pickKey(category, isVeg) (new dishes have no photograph), the menu is re-rendered, and a toast confirms the addition. A live preview of the artwork appears as soon as a category and dietary type are chosen.

```
addDishForm.addEventListener("submit", (e) => {
  e.preventDefault();
  const name = $("#adName").value.trim();
  const category = $("#adCategory").value;
  const price = parseInt($("#adPrice").value, 10);
  const vegInput = document.querySelector('input[name="adVeg"]:checked');
  const desc = $("#adDesc").value.trim();

  let valid = true;

  function setErr(id, msg) {
    const el = document.getElementById(id);
    if (!el) return;
    el.textContent = msg;
    const field = el.closest(".field");
    if (field) field.classList.toggle("has-error", Boolean(msg));
    valid = valid && !msg;
  }

  setErr("adNameError",     !name             ? "Enter a dish name."           :
     name.length < 2 ? "Name must be at least 2 characters." : "");
  setErr("adCategoryError", !category         ? "Select a category."           :
     "");
  setErr("adPriceError",    !price || price < 1 ? "Enter a valid price in ₹."    :
     "");
  setErr("adVegError",      !vegInput         ? "Select vegetarian or
     non-vegetarian." : "");
  setErr("adDescError",     !desc             ? "Enter a short description."     :
     "");

  if (!valid) return;
```

6.6 Reservation Form and Validation

The reservation form contains six required fields and one optional field (special requests, limited to 300 characters with a live counter). Time options are generated in a loop, and the date input is limited to the range between today and 90 days ahead. Each field has a validator that returns an error message or an empty string. Validation runs when the user leaves a field, re-runs on every keystroke once an error is shown, and runs for all fields on submit; the first invalid field receives focus and an error toast is displayed.

```
const validators = {
  name(v) {
    if (!v) return "Enter your full name.";
    if (v.length < 2) return "Name must be at least 2 characters.";
    if (!/^[\p{L}][\p{L} .'-]*$/u.test(v)) return "Name can only contain letters,
       spaces, dots, hyphens and apostrophes.";
    return "";
  },
  email(v) {
    if (!v) return "Enter your email address.";
    if (!/^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/.test(v)) return "Enter a valid email
```

```
      address, like name@example.com.";
    return "";
  },
  phone(v) {
    if (!v) return "Enter your phone number.";
    if (!/^\+?[\d\s()-]+$/.test(v)) return "Phone number can only contain digits,
      spaces, + and hyphens.";
    const digits = v.replace(/\D/g, "");
    if (digits.length < 10 || digits.length > 13) return "Enter a phone number with
      10 to 13 digits.";
    return "";
  },
  guests(v) { return !v ? "Select the number of guests." : ""; },
  date(v) {
    if (!v) return "Choose a reservation date.";
    if (v < todayISO) return "That date has passed. Choose today or a future date.";
    if (v > fields.date.max) return "Reservations open up to 90 days ahead.";
    return "";
  },
  time(v) {
    if (!v) return "Select a preferred time.";
    if (fields.date.value === todayISO) {
      const [h, mn] = v.split(":").map(Number);
      const now = new Date();
      if (h * 60 + mn <= now.getHours() * 60 + now.getMinutes()) return "That time
        has already passed. Choose a later time.";
    }
    return "";
  }
};
```

On success, the form is hidden and a confirmation panel shows a personalised summary (guest's first name, party size, long-format date and time). A "Make another reservation" button clears all state and restores the form.

```
form.addEventListener("submit", (e) => {
  e.preventDefault();
  const results = Object.keys(fields).map((key) => [key, validateField(key)]);
  const firstInvalid = results.find(([, ok]) => !ok);

  if (firstInvalid) {
    fields[firstInvalid[0]].focus();
    showToast("Some details are missing or incorrect. Check the highlighted
      fields.", "error");
    return;
  }

  const guests = Number(fields.guests.value);
  const timeLabel = fields.time.options[fields.time.selectedIndex].text;
  const firstName = fields.name.value.trim().split(" ")[0];

  $("#confirmText").textContent =
    `Thank you, ${firstName}. We have a table for ${guests} ${guests === 1 ? "guest"
      : "guests"} on ${formatLongDate(fields.date.value)} at ${timeLabel}.`;

  form.hidden = true;
  confirmation.hidden = false;
  confirmation.focus();
  showToast("Reservation confirmed. We look forward to seeing you! □", "success");
});
```

6.7 Accessibility Features

- A skip link, a single h1, landmark regions and a logical heading hierarchy
- Labels bound to every input, error messages with role="alert", and aria-invalid / aria-describedby set on invalid fields

- aria-pressed on filter buttons, aria-expanded and aria-label on the navigation toggle, aria-live on the menu grid and toast region
- Descriptive aria-labels on icon-only buttons (close, back to top, social links) and role="img" with an aria-label on the veg / non-veg markers
- Visible :focus-visible outlines, native dialog focus handling and focus restoration
- prefers-reduced-motion support and a <noscript> fallback

6.8 Navigation, Scroll Behaviour and Notifications

A throttled scroll handler (using requestAnimationFrame) makes the header solid after 40 px, reveals the back-to-top button after 600 px and computes which section is current for the scroll-spy. An IntersectionObserver fades sections into view the first time they are seen. On narrow screens the hamburger button toggles a full-screen menu, locks body scrolling, closes on link click or Escape and closes automatically when the viewport grows past 860 px. Toasts are created on demand, announced politely to screen readers and removed after five seconds or when dismissed.

6.9 Dish Photographs and the Artwork Module (art.js)

Version 2 of the site shows a real photograph for each of the 24 menu dishes, stored in the images folder and referenced by the image field of each MENU entry. Because these files are local, they load without an internet connection. To guarantee that every dish still has a picture that matches the site's palette — for example, a dish added at run time that has no photograph — art.js draws a dark, warm-toned SVG plate for each dish (skewers, curries, biryani, desserts, drinks and so on) using shared building blocks such as plates, leaves, lemon wedges and a seeded pseudo-random generator for garnish. SavorArt.uri(key) returns a cached data URI that can be used directly as an image source, and SavorArt.pickKey(category, veg) chooses a suitable illustration for dishes added at run time. If an image still fails to load, a global error handler adds a class that shows a styled placeholder with the dish name.

6.10 Functional Modules

1. Content and Layout Module

- Semantic page structure, responsive grids and the design-token stylesheet
- Hero, Featured, About, Contact and Footer sections

2. Menu Module

- MENU data array, card and featured renderers, category and diet filters
- Dish details dialog and the Add Dish form with live artwork preview

3. Reservation Module

- Guest and time drop-downs, date window, field validators and character counter
- Confirmation view and reset

4. Navigation and Feedback Module

- Sticky header, mobile drawer, scroll-spy, back-to-top button, reveal animations and toast notifications

5. Imagery Module

- Dish photographs from the images folder, SVG fallback illustrations, data-URI caching, category-based picker and image-error fallback

7. Results and Discussion

7.1 Overview of the Delivered Site

The final deliverable is a complete restaurant website of about 1,900 lines of source code across four files — index.html (390 lines), style.css (489 lines), script.js (721 lines) and art.js (310 lines) — plus a 1,916-line standalone build and 24 dish photographs. The screenshots below were captured from the running site in Chromium with external network requests blocked, which also demonstrates the offline behaviour: fonts fall back to system serif and sans-serif faces and the remote hero photograph is absent, yet the layout, the local dish photographs and all functionality remain intact.

7.2 Home and Featured Dishes

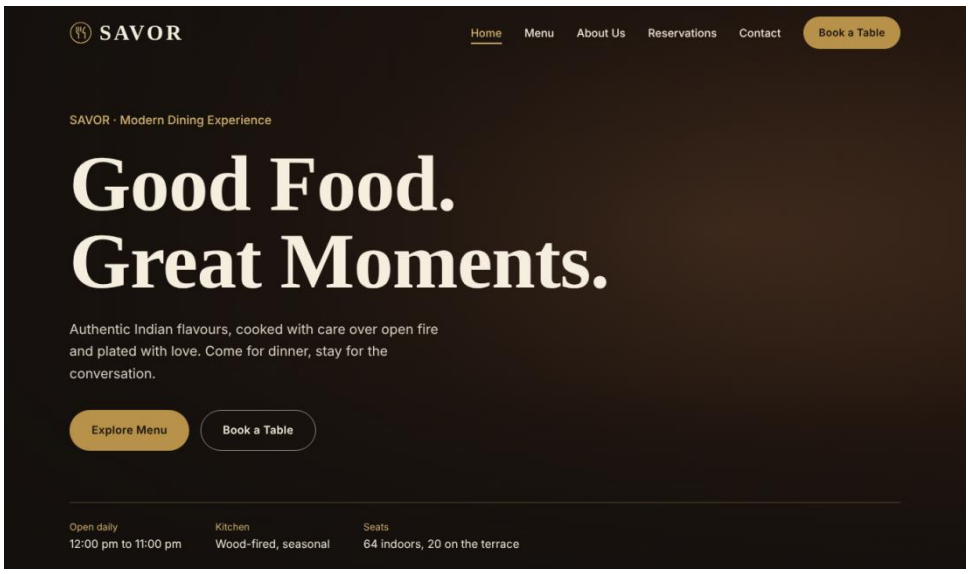


Fig 7.1 — Home section with hero text, calls to action and key facts (desktop, remote photograph not loaded)

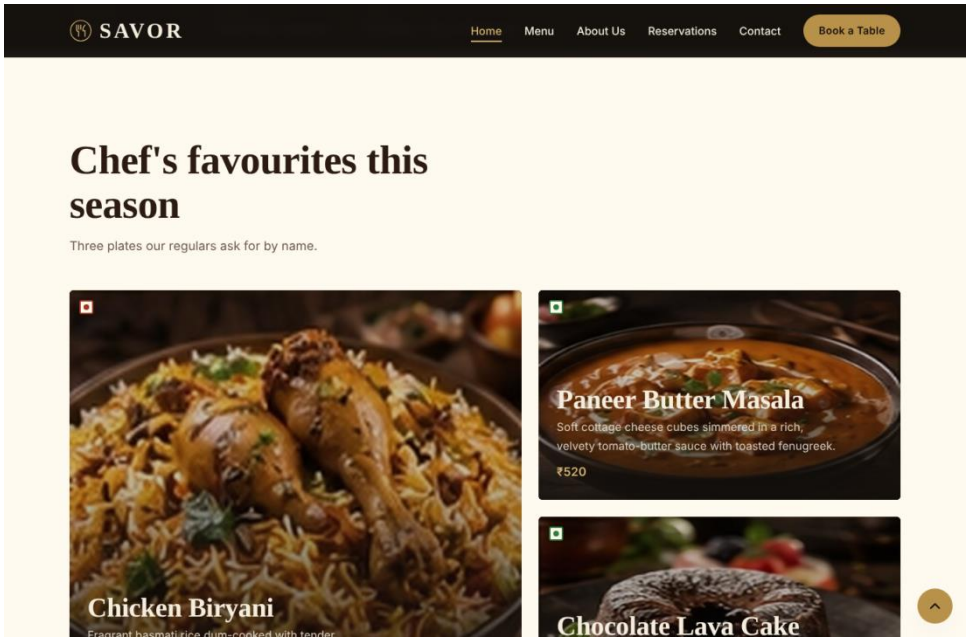


Fig 7.2 — "Chef's favourites this season" featured cards with veg/non-veg markers and prices

7.3 Menu, Filters and Dish Details

Filtering is instantaneous because it re-renders from the in-memory array. Figure 7.3 shows the combination Main Course + Non-Veg Only, which returns three dishes. Figure 7.4 shows the dish-details dialog opened for Spicy Chicken Wings, with category, dietary type, description, ingredients, a serving note, the price and a shortcut to the reservation form.

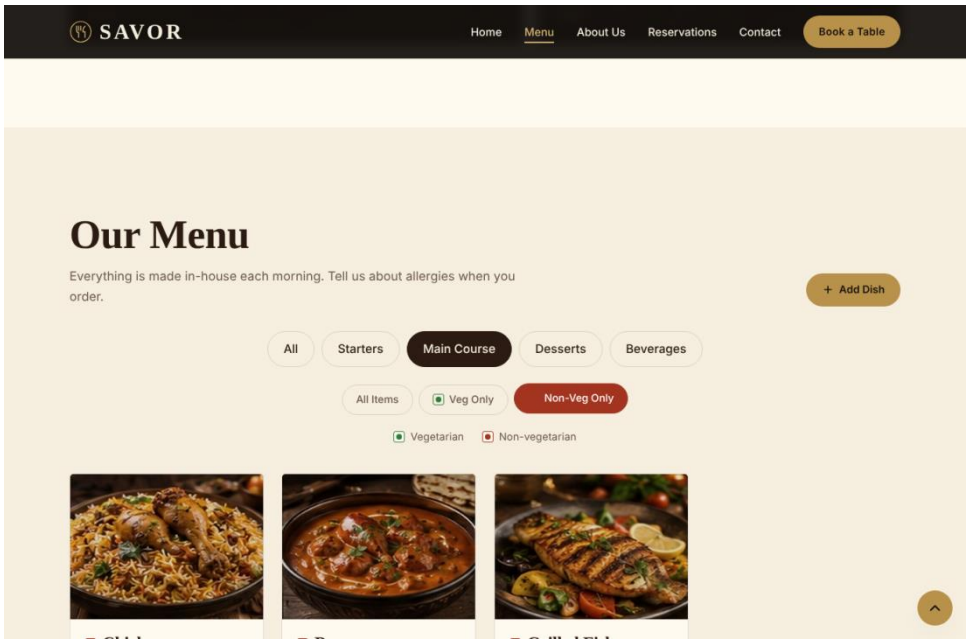


Fig 7.3 — Menu filtered by Main Course and Non-Veg Only

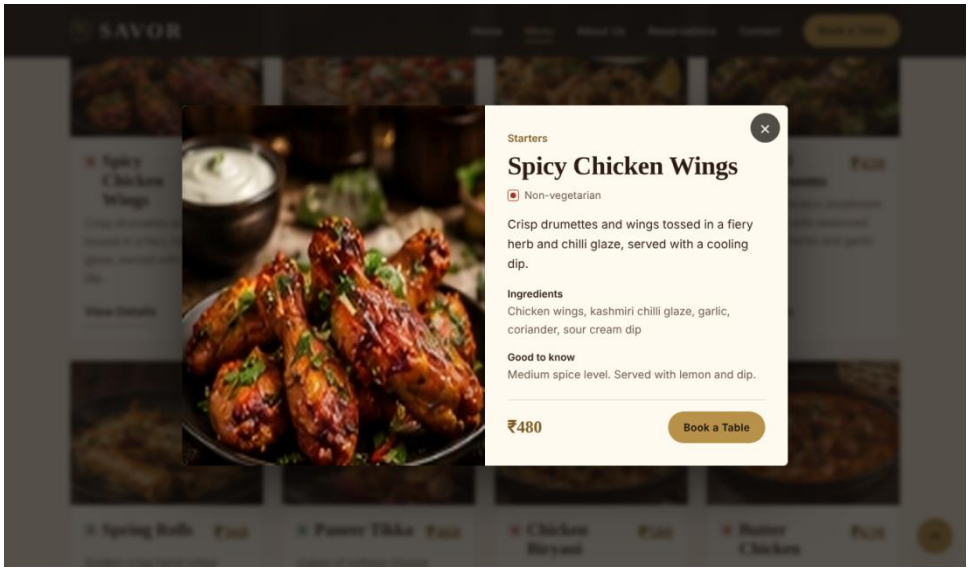


Fig 7.4 — Dish details dialog for Spicy Chicken Wings

7.4 Add Dish Form

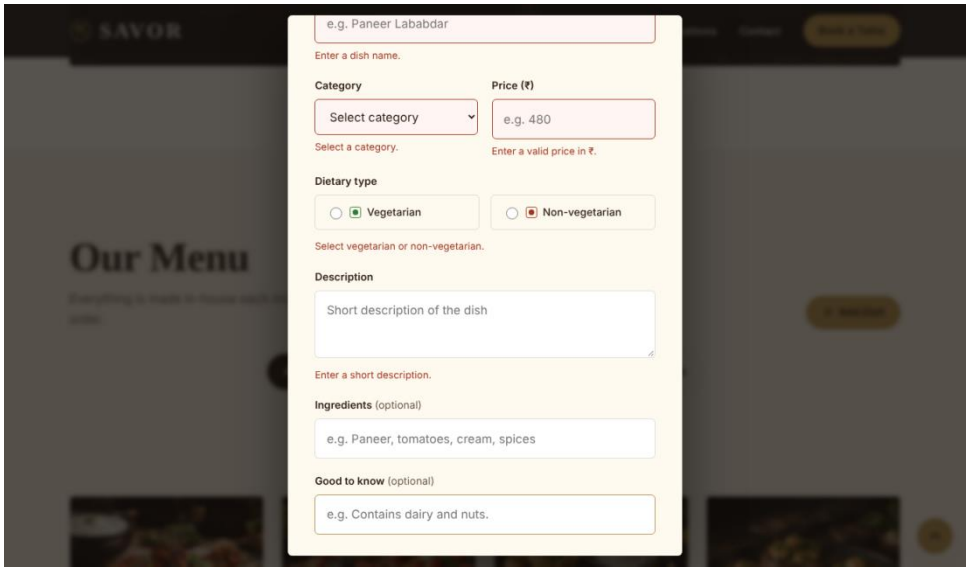


Fig 7.5 — Add Dish dialog showing validation messages for an empty submission

7.5 Reservation Form

Submitting the empty form displays an inline message beneath every required field together with an error toast and moves focus to the first invalid field (Figure 7.6). After valid details are entered, a confirmation panel summarises the booking (Figure 7.7).

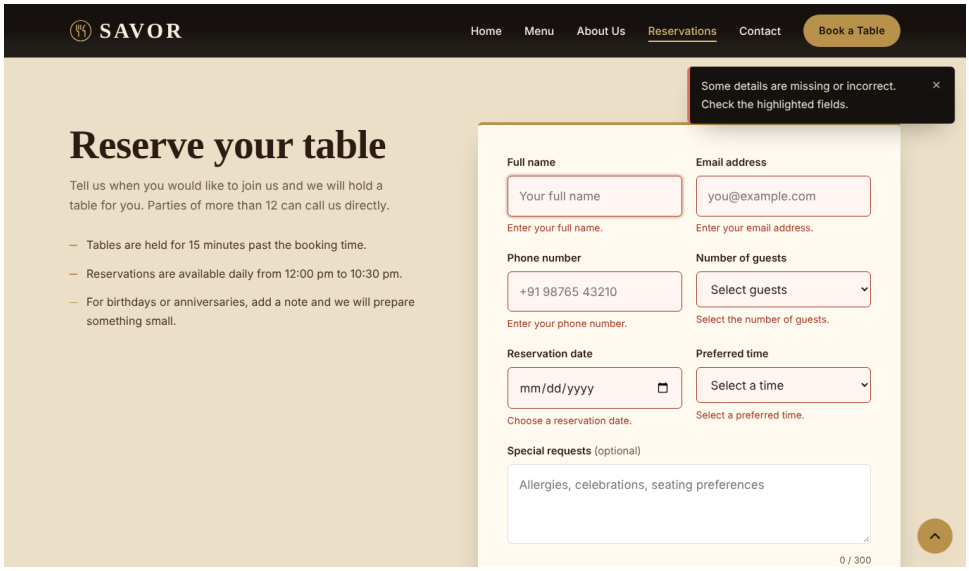


Fig 7.6 — Reservation form showing field-level errors and an error toast

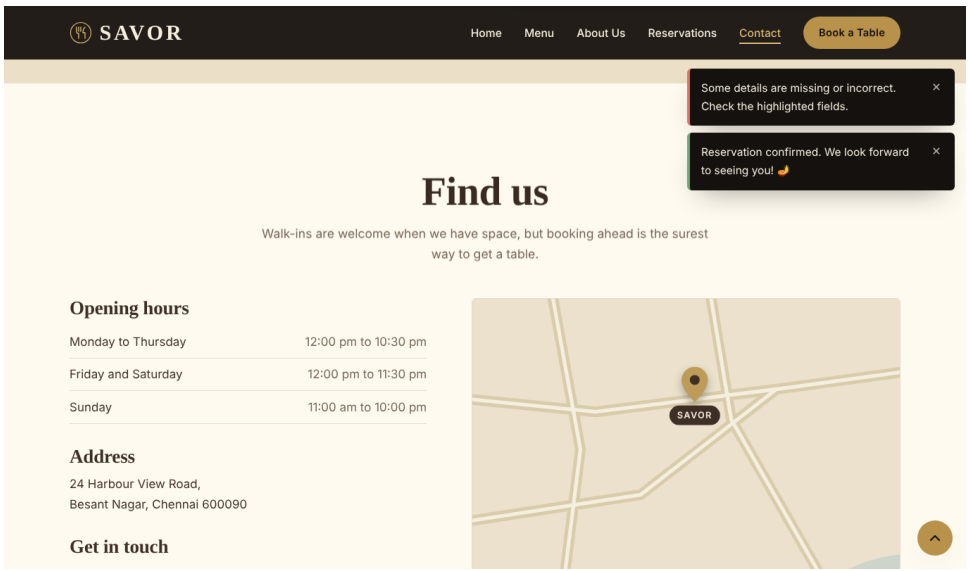


Fig 7.7 — Reservation confirmation panel with personalised summary

7.6 Mobile Experience

At 390 px width the layout becomes a single column, the navigation collapses into a hamburger button that opens a full-screen drawer, and no horizontal scrolling occurs (verified programmatically). Selecting a link closes the drawer and scrolls to the section.

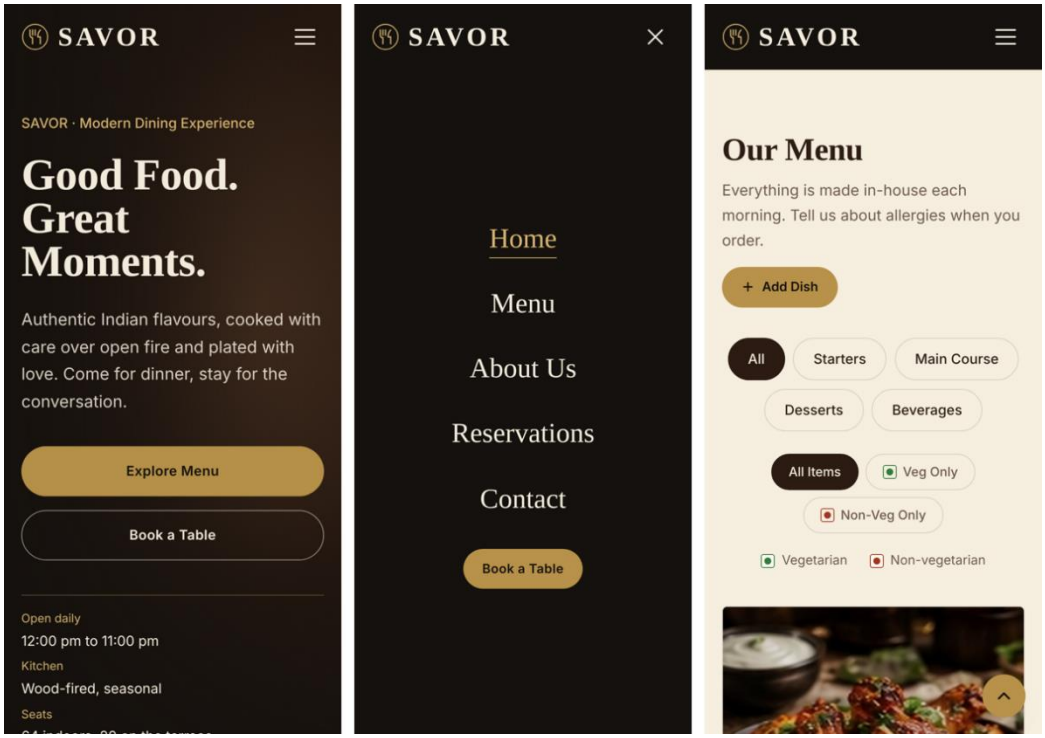


Fig 7.8 — Mobile layout: home, navigation drawer and menu (390 px wide)

7.7 Summary of Results

Aspect	Target	Observed
Menu size	24 dishes in four categories	6 starters, 6 mains, 6 desserts, 6 beverages
Diet split	Veg and non-veg marked on every dish	19 vegetarian, 5 non-vegetarian
Featured dishes	3 highlighted dishes	3 rendered from featured: true
Reservation options	Party size 1–12; 30-minute slots	12 guest options; 22 slots (12:00 pm–10:30 pm)
Responsive breakpoints	Desktop, tablet, phone	992 px, 860 px and 640 px; no horizontal scroll at 390 px
JavaScript errors	None	No uncaught page errors during the test run
Dish images	Photograph on every dish card	24 of 24 menu photographs loaded from the local images folder
Offline behaviour	Usable without internet	Layout, photographs, filters, dialogs and forms work; only fonts and the hero/About photos need the internet

8. Challenges and Issues Faced

Building the site raised two kinds of challenge: technical problems that were solved in the implementation, and content or design inconsistencies that were identified during review and remain as recommendations for the next iteration.

8.1 Images That Depend on the Network

- The first version used food photographs hosted on third-party servers, which may fail to load and leave empty cards and a poor first impression

Resolution: the menu photographs are now bundled in the images folder and referenced with relative paths, so they load offline. art.js still draws a matching SVG illustration as a fallback for any dish without a photograph, and a capture-phase error listener marks any broken image's wrapper with an img-failed class, which displays a styled placeholder with the dish name. Only the hero and About photographs still come from Unsplash.

8.2 Combining Two Independent Filters

- Category and vegetarian/non-vegetarian filters had to work together without conflicting states or stale buttons

Resolution: the two selections are stored in separate variables and applied together inside a single renderMenu() function; button states are updated through aria-pressed and classes, and an empty-result message is shown when a combination has no dishes (for example, Desserts + Non-Veg).

8.3 Safely Displaying User-Entered Content

- The Add Dish form lets a user type text that is then displayed on the page, which could allow HTML injection

Resolution: all dynamic text is passed through an escapeHTML() helper before being placed in a template, and the dish dialog and toasts use textContent or escaped strings. No user input is inserted as raw HTML.

8.4 Date and Time Validation

- Using UTC-based date conversion can make "today" wrong near midnight; same-day bookings need to reject times that have already passed; and changing the date can invalidate a previously chosen time

Resolution: dates are formatted from local year, month and day values; the minimum and maximum dates are set on the input (today to +90 days); the time validator compares against the current time when the chosen date is today; and a change of date re-validates the time field.

8.5 Mobile Navigation Overlay

- A full-screen fixed menu inside a header that uses backdrop-filter was sized to the header instead of the screen

Resolution: backdrop-filter is disabled at mobile widths and the header is given a solid background, so the fixed drawer fills the viewport. Body scrolling is locked while the menu is open, and the drawer closes on Escape, on link click and when the viewport becomes wide.

8.6 Inconsistent Opening Hours

- The hero says "Open daily 12:00 pm to 11:00 pm", the Contact table lists different hours for Friday–Saturday (to 11:30 pm) and Sunday (11:00 am to 10:00 pm), and the booking form always offers 12:00 pm to 10:30 pm

Recommendation: keep one hours definition in the script and use it to generate the hero text, the table and the time slots for the selected weekday, so that no one can book a time when the restaurant is closed.

8.7 No Server-Side Storage or E-mail

- The confirmation panel says a confirmation "has been noted for the email address you provided", but the project has no backend, so nothing is stored or sent. Dishes added through the Add Dish form also disappear when the page is reloaded

Recommendation: either label the site clearly as a demonstration or connect the form to a service (for example, a serverless function or form-handling provider) and persist added dishes with localStorage or a database.

8.8 Menu Content Consistency

- The site presents itself as "seasonal Indian" cuisine, but the menu now mixes in international dishes such as Bruschetta, Calamari, Tiramisu and Cheesecake, and the Mojito is described as a virgin (non-alcoholic) drink; dishes added at run time have no photograph and reuse one illustration per category

Recommendation: align the tagline and About text with the actual menu (or the menu with the concept) and let new dishes upload their own photograph.

9. Testing

The site was tested with 20 automated checks written with Playwright and run in headless Chromium against the local files, with external network requests blocked. Desktop checks used a 1366 × 800 viewport and mobile checks used a 390 × 844 touch viewport. The same checks were supported by manual review of the screenshots in Chapter 7. The results are summarised below.

No.	Test case	Expected outcome	Result	Status
1	Load index.html	No uncaught JavaScript errors; 24 menu cards (with 24 local photographs loaded) and 3 featured cards rendered	As expected	Pass
2	Category filters	Starters 6, Main Course 6, Desserts 6, Beverages 6, All 24	As expected	Pass
3	Veg Only / Non-Veg Only	19 vegetarian and 5 non-vegetarian dishes	19 / 5	Pass

No.	Test case	Expected outcome	Result	Status
4	Combined filters	Main Course + Non-Veg shows 3 dishes; Desserts + Non-Veg shows “No dishes match your selection.”	3 / message shown	Pass
5	View Details	Dialog opens for the first dish (Spicy Chicken Wings, ₹480)	As expected	Pass
6	Close dialog with Escape	Dialog closes	Closed	Pass
7	Add Dish with empty form	Five error messages displayed	5 messages	Pass
8	Add valid dish	Card count rises from 24 to 25 and a success toast appears	25 and toast	Pass
9	Reload after adding	Added dish is not kept (in-memory data)	24 cards	Known limitation
10	Reservation, empty submit	Six field errors shown with error toast	6 messages	Pass
11	Invalid name, e-mail, phone	Specific messages for each field	As expected	Pass
12	Past date and date beyond 90 days	“That date has passed…” and “Reservations open up to 90 days ahead.”	As expected	Pass
13	Valid reservation	Confirmation panel with summary; form hidden	Summary shown	Pass
14	Make another reservation	Form is restored and cleared	Restored	Pass
15	Drop-down options	12 guest options; 22 time slots from 12:00 pm to 10:30 pm	12 / 22	Pass
16	Character counter	Shows 39 / 300 for a 39-character request	39 / 300	Pass
17	Mobile navigation (390 px)	Hamburger visible; drawer opens; closes after link click	As expected	Pass
18	Horizontal overflow (390 px)	No horizontal scrollbar	None	Pass
19	Keyboard: first Tab stop	Skip link receives focus	Skip link	Pass
20	Standalone build	savor-standalone.html renders 24 menu cards	24 cards	Pass

All tests passed except test 9, which confirms the documented limitation that added dishes are not persisted. Cross-browser testing on Firefox and Safari, automated accessibility audits and user-acceptance testing are listed under future work.

Appendix A — Menu Data Model (script.js excerpt)

The first entries of the MENU array, which drives every dish view on the site. The complete array holds 24 dishes.

```
/* ---------- Menu data (Indian dishes) ---------- */
const CATEGORY_LABELS = {
  starters: "Starters",
  mains: "Main Course",
  desserts: "Desserts",
  beverages: "Beverages"
};

/* artKey maps to window.SavorArt.uri(key) from art.js */
const MENU = [
  /* ── Starters (Dishes 1 – 6) ──────────────────────────── */
  {
    id: 1, category: "starters", veg: false, featured: false, price: 480,
    name: "Spicy Chicken Wings",
    desc: "Crisp drumettes and wings tossed in a fiery herb and chilli glaze, served
       with a cooling dip.",
    ingredients: "Chicken wings, kashmiri chilli glaze, garlic, coriander, sour
       cream dip",
    note: "Medium spice level. Served with lemon and dip.",
    image: "images/01_spicy_chicken_wings.jpg",
    artKey: "chickenMalai"
  },
  {
    id: 2, category: "starters", veg: true, featured: false, price: 380,
    name: "Bruschetta",
    desc: "Toasted artisan sourdough rubbed with garlic, topped with heirloom
       tomatoes, basil and olive oil.",
    ingredients: "Sourdough bread, heirloom tomatoes, fresh basil, garlic, extra
       virgin olive oil",
    note: "Vegetarian and dairy-free.",
    image: "images/02_bruschetta.jpg",
    artKey: "haraBhara"
  },
  {
    id: 3, category: "starters", veg: false, featured: false, price: 540,
    name: "Calamari",
    desc: "Tender calamari rings dusted in seasoned crumb, flash-fried golden and
       served with garlic aioli.",
    ingredients: "Fresh squid rings, cracked black pepper, garlic aioli, lemon
       wedge",
    note: "Contains shellfish. Served with lemon wedge.",
    image: "images/03_calamari.jpg",
    artKey: "fishFry"
  },
```

Appendix B — Reservation Form Markup (index.html excerpt)

Semantic markup for the reservation form, showing labels, autocomplete hints, error placeholders with role="alert" and the confirmation panel.

```
        <form id="reservationForm" novalidate>
          <div class="form-grid">
            <div class="field">
              <label for="name">Full name</label>
              <input type="text" id="name" name="name" autocomplete="name"
                 placeholder="Your full name" required>
              <p class="error-msg" id="nameError" role="alert"></p>
            </div>
            <div class="field">
              <label for="email">Email address</label>
              <input type="email" id="email" name="email" autocomplete="email"
```

```
            placeholder="you@example.com" required>
          <p class="error-msg" id="emailError" role="alert"></p>
        </div>
        <div class="field">
          <label for="phone">Phone number</label>
          <input type="tel" id="phone" name="phone" autocomplete="tel"
            placeholder="+91 98765 43210" required>
          <p class="error-msg" id="phoneError" role="alert"></p>
        </div>
        <div class="field">
          <label for="guests">Number of guests</label>
          <select id="guests" name="guests" required>
            <option value="">Select guests</option>
          </select>
          <p class="error-msg" id="guestsError" role="alert"></p>
        </div>
        <div class="field">
          <label for="date">Reservation date</label>
          <input type="date" id="date" name="date" required>
          <p class="error-msg" id="dateError" role="alert"></p>
        </div>
        <div class="field">
          <label for="time">Preferred time</label>
          <select id="time" name="time" required>
            <option value="">Select a time</option>
          </select>
          <p class="error-msg" id="timeError" role="alert"></p>
        </div>
        <div class="field field-full">
          <label for="requests">Special requests <span
            class="optional">(optional)</span></label>
          <textarea id="requests" name="requests" rows="3" maxlength="300"
            placeholder="Allergies, celebrations, seating preferences"></textarea>
          <p class="char-count" id="charCount">0 / 300</p>
        </div>
      </div>
      <button type="submit" class="btn btn-gold btn-block">Confirm
        Reservation</button>
    </form>

    <div class="confirmation" id="confirmation" hidden tabindex="-1">
      <svg viewBox="0 0 48 48" class="check" aria-hidden="true"><circle cx="24"
        cy="24" r="22" fill="none" stroke="currentColor" stroke-width="2"/><path
        d="M14 25l7 7 13-15" fill="none" stroke="currentColor" stroke-width="3"
        stroke-linecap="round" stroke-linejoin="round"/></svg>
      <p class="res-badge">CONFIRMED</p>
      <h3>Your table is reserved</h3>
      <p id="confirmText"></p>
      <p class="confirm-sub">A confirmation has been noted for the email address
        you provided.</p>
      <button type="button" class="btn btn-outline-dark" id="newBooking">Make
        another reservation</button>
    </div>
  </div>
```

Appendix C — Design Tokens and Responsive Rules (style.css excerpt)

```
:root {
  --cream: #f6efe0;
  --cream-deep: #ebdfc7;
  --paper: #fffaf0;
  --brown: #2b1b12;
  --brown-soft: #4a3326;
  --black: #15110e;
  --gold: #b8924a;
  --gold-light: #d8b774;
```

```
  --gold-text: #7d5f24;      /* gold dark enough for text on cream */
  --muted: #6b584a;
  --line: rgba(43, 27, 18, 0.14);
  --danger: #a3341f;
  --success: #3d6b45;

  --font-display: "Cormorant Garamond", Georgia, "Times New Roman", serif;
  --font-body: "Hanken Grotesk", "Segoe UI", system-ui, sans-serif;

  --radius: 6px;
  --shadow: 0 18px 40px -20px rgba(21, 17, 14, 0.45);
  --header-h: 76px;
  --ease: cubic-bezier(.2, .7, .2, 1);
}


@media (max-width: 992px) {
  .featured-grid { grid-template-columns: 1fr 1fr; grid-template-rows: auto;
      min-height: 0; }
  .feature-card:first-child { grid-column: 1 / -1; grid-row: auto; min-height: 420px;
      }
  .about-grid, .res-grid, .contact-grid { grid-template-columns: 1fr; }
  .res-intro { position: static; }
  .about-media { max-width: 560px; }
  .footer-grid { grid-template-columns: 1fr 1fr; }
  .footer-brand { grid-column: 1 / -1; }
}
  ...
@media (prefers-reduced-motion: reduce) {
  html { scroll-behavior: auto; }
  *, *::before, *::after { animation-duration: .01ms !important;
      animation-iteration-count: 1 !important; transition-duration: .01ms !important; }
  .js .reveal { opacity: 1; transform: none; }
}
```

Appendix D — Interaction Logic (script.js excerpts)

Add Dish Submission

```
  addDishForm.addEventListener("submit", (e) => {
    e.preventDefault();
    const name = $("#adName").value.trim();
    const category = $("#adCategory").value;
    const price = parseInt($("#adPrice").value, 10);
    const vegInput = document.querySelector('input[name="adVeg"]:checked');
    const desc = $("#adDesc").value.trim();

    let valid = true;

    function setErr(id, msg) {
      const el = document.getElementById(id);
      if (!el) return;
      el.textContent = msg;
      const field = el.closest(".field");
      if (field) field.classList.toggle("has-error", Boolean(msg));
      valid = valid && !msg;
    }

    setErr("adNameError",     !name              ? "Enter a dish name."           :
        name.length < 2 ? "Name must be at least 2 characters." : "");
    setErr("adCategoryError", !category          ? "Select a category."           :
        "");
    setErr("adPriceError",    !price || price < 1 ? "Enter a valid price in ₹."    :
        "");
    setErr("adVegError",      !vegInput          ? "Select vegetarian or
        non-vegetarian." : "");
    setErr("adDescError",     !desc              ? "Enter a short description."    :
        "");
```

```
  if (!valid) return;

  const isVeg = vegInput.value === "veg";
  const key = window.SavorArt
    ? window.SavorArt.pickKey(category, isVeg)
    : (isVeg ? "haraBhara" : "muttonSeekh");

  const newDish = {
    id: nextId++,
    category,
    veg: isVeg,
    featured: false,
    price,
    name,
    desc,
    ingredients: $("#adIng").value.trim() || "Ask your server.",
    note: $("#adNote").value.trim() || "Made fresh to order.",
    artKey: key
  };

  MENU.push(newDish);
  closeAddDish();
  renderMenu();
  showToast(`"${name}" has been added to the menu.`, "success");

  /* If the new dish matches a current filter, scroll the grid into view */
  if (currentCategory === "all" || currentCategory === newDish.category) {
    menuGrid.scrollIntoView({ behavior: "smooth", block: "nearest" });
  }
});
```

Mobile Navigation, Scroll-Spy and Back-to-Top

```
/* ---------- Header, mobile nav, scroll spy ---------- */
const siteHeader = $("#siteHeader");
const navToggle = $("#navToggle");
const nav = $("#primaryNav");
const navLinks = $$(".nav-link");

function setNav(open) {
  nav.classList.toggle("is-open", open);
  siteHeader.classList.toggle("menu-open", open);
  navToggle.setAttribute("aria-expanded", String(open));
  navToggle.setAttribute("aria-label", open ? "Close menu" : "Open menu");
  document.body.style.overflow = open ? "hidden" : "";
}

navToggle.addEventListener("click", () =>
    setNav(!nav.classList.contains("is-open")));
$$("a", nav).forEach((a) => a.addEventListener("click", () => setNav(false)));
document.addEventListener("keydown", (e) => {
  if (e.key === "Escape" && nav.classList.contains("is-open")) { setNav(false);
      navToggle.focus(); }
});
window.matchMedia("(min-width: 861px)").addEventListener("change", (e) => { if
    (e.matches) setNav(false); });

const sectionIds = ["home", "menu", "about", "reservations", "contact"];
const spyTargets = sectionIds.map((id) => document.getElementById(id));
const toTop = $("#toTop");

function onScroll() {
  siteHeader.classList.toggle("is-solid", window.scrollY > 40);
  toTop.hidden = window.scrollY < 600;
  const probe = window.scrollY + window.innerHeight * 0.35;
  let current = "home";
  spyTargets.forEach((section) => { if (section && section.offsetTop <= probe)
      current = section.id; });
  navLinks.forEach((link) => link.classList.toggle("is-active",
      link.getAttribute("href") === `#${current}`));
}
```

```
let ticking = false;
window.addEventListener("scroll", () => {
  if (!ticking) { requestAnimationFrame(() => { onScroll(); ticking = false; });
      ticking = true; }
}, { passive: true });
onScroll();

toTop.addEventListener("click", () => window.scrollTo({ top: 0, behavior: "smooth"
    }));
```

Reservation Form Setup

```
/* Populate guests dropdown (1–12) */
for (let i = 1; i <= 12; i++) {
  fields.guests.add(new Option(i === 1 ? "1 guest" : `${i} guests`, i));
}

/* Time slots every 30 min from 12:00 pm to 10:30 pm */
for (let m = 12 * 60; m <= 22 * 60 + 30; m += 30) {
  const h24 = Math.floor(m / 60);
  const min = m % 60;
  const label = `${h24 % 12 === 0 ? 12 : h24 % 12}:${String(min).padStart(2, "0")}
      ${h24 >= 12 ? "pm" : "am"}`;
  fields.time.add(new Option(label, `${String(h24).padStart(2,
      "0")}:${String(min).padStart(2, "0")}`));
}

/* Date bounds */
const pad = (n) => String(n).padStart(2, "0");
const toISO = (d) => `${d.getFullYear()}-${pad(d.getMonth() +
    1)}-${pad(d.getDate())}`;
const today = new Date();
const todayISO = toISO(today);
const maxDate = new Date(today); maxDate.setDate(maxDate.getDate() + 90);
fields.date.min = todayISO;
fields.date.max = toISO(maxDate);
```

10. Conclusion and Future Scope

10.1 Conclusion

SAVOR demonstrates that a complete, attractive and accessible restaurant website can be built with only HTML, CSS and JavaScript. The data-driven menu, combined filters, dish-details dialog, Add Dish form, reservation validation and responsive navigation work together as a coherent experience, and 19 of the 20 verification checks passed, with the twentieth confirming a documented limitation. The use of design tokens, a single data array and small, well-named functions keeps the code easy to read and modify, and the bundled photographs, backed by the SVG artwork fallback, mean the menu looks complete even without network access.

The project also provided practical experience with semantic HTML, responsive layout, the DOM and event delegation, form validation, accessibility techniques and defensive handling of user input, and showed the value of reviewing a finished product for content consistency as well as for code correctness.

10.2 Known Limitations

- Reservations are validated and confirmed on screen but are not transmitted, stored or e-mailed, because there is no backend
- Dishes added through the Add Dish form exist only in memory and are lost on reload; they also have no photograph and share one fallback illustration per category
- Opening hours are written in several places and are not fully consistent; the booking form does not adapt to the weekday
- The hero and About photographs and web fonts require an internet connection; social links are placeholders and the map is a drawn illustration
- Table availability is not checked, so the same slot can be requested repeatedly
- Testing was automated for Chromium only; Firefox, Safari and screen-reader testing have not yet been carried out

10.3 Future Enhancements

Several enhancements can be incorporated to further improve the system:

- Connect the reservation form to a backend or serverless function with e-mail/SMS confirmation and table-availability checks
- Persist menu changes using localStorage or a database, add edit and delete actions and protect the Add Dish function behind staff authentication
- Define opening hours once and generate hero text, the hours table and time slots for each weekday from it
- Allow photograph upload for each dish and add a marker for alcoholic items if any are added to the menu
- Embed an interactive map, add an online-ordering flow and support multiple languages
- Add search, allergen filters and sorting to the menu

- Run cross-browser, automated accessibility (for example, axe) and performance (Lighthouse) audits and add them to a continuous-integration workflow

11. References

- MDN Web Docs — HTML, CSS and JavaScript reference, Mozilla. https://developer.mozilla.org
- WHATWG — HTML Living Standard, the dialog element. https://html.spec.whatwg.org
- W3C — Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/
- W3C WAI — WAI-ARIA Authoring Practices Guide. https://www.w3.org/WAI/ARIA/apg/
- W3C — CSS Grid Layout and CSS Flexible Box Layout Modules. https://www.w3.org/TR/css-grid-1/ and https://www.w3.org/TR/css-flexbox-1/
- Marcotte, E. — "Responsive Web Design", A List Apart, May 2010.
- Google Fonts — Cormorant Garamond and Hanken Grotesk typefaces. https://fonts.google.com
- Unsplash — Free photographs used for the hero and About sections. https://unsplash.com
- Playwright documentation — browser automation and testing. https://playwright.dev

12. System Usage Use Case Description

The system has two actors: the Guest, who browses the site and requests a table, and the Staff member, who adds dishes to the menu through the Add Dish form. Every interaction runs in the browser.

Use Case	Description
Browse menu	Guest scrolls to the Menu section or follows the Menu link and views dish cards with image, name, marker, price and description.
Filter menu	Guest selects a category and/or Veg Only or Non-Veg Only; the grid updates immediately and shows a message if no dish matches.
View dish details	Guest selects View Details or a featured card; a dialog shows ingredients, a serving or allergen note and the price, with a Book a Table shortcut.
Add dish	Staff member opens Add Dish, enters name, category, price, dietary type and description (ingredients and note optional) and submits; the dish appears in the menu and a toast confirms.
Reserve a table	Guest completes the reservation form; the system validates every field and shows a confirmation summary with date, time and party size.
Navigate	Guest uses the header links, the Book a Table button, the mobile drawer or the back-to-top button; the active section is highlighted.
Find the restaurant	Guest reads opening hours, address, phone and e-mail in the Contact section.

Because all processing happens in the browser, no installation, account or server is needed for any of these use cases.

12.1 Comparison with Traditional Methods

Criterion	Printed menu and telephone booking	SAVOR website (current version)
Menu updates	Menus must be reprinted	Edit one MENU array
Dietary guidance	Guest must ask staff	Veg/non-veg markers, filters and allergen notes
Dish information	Name and price only	Description, ingredients, note and image
Reservation	Phone call during opening hours	Online form with validation (request is not yet transmitted)
Reach	Guests who are present or can call	Any device with a browser
Cost and infrastructure	Printing and staff time	Static files; no server required

12.2 Comparison of the Two Delivered Versions

Aspect	Separate files (index.html, css, js)	Single file (savor-standalone.html)
Structure	HTML, CSS and JavaScript separated by concern	All code inlined in one 1,916-line file (dish photographs remain in the images folder)
Maintainability	Edit one file per concern; browser caches CSS/JS	Convenient to share but code is duplicated and may drift from the source
Portability	Folder structure must be kept (including js/art.js and the images folder)	Needs only the images folder beside it; otherwise self-contained
Best use	Development and further customisation	Quick demonstration or submission

13. User Manual Running the Application

The following steps describe how to open and use the SAVOR website.

Step 1: Prepare the Files

Unzip the project folder completely; do not open files from inside the zip. Any current version of Chrome, Edge or Firefox may be used. No installation is required.

Step 2: Project Structure

Keep the folder structure exactly as shown:

```
savor/
  index.html            main page
  css/style.css         all styles
  js/art.js             fallback dish artwork generator
  js/script.js          menu data and interactions
  images/               24 dish photographs (01_...jpg to 24_...jpg)
  savor-standalone.html single-file version
  README.txt
```

Step 3: Open the Site

Double-click index.html (or savor-standalone.html). Optionally, serve the folder locally:

```
cd savor
python -m http.server 8000
# then open http://localhost:8000
```

Step 4: Use the Site

- Use the header links or the Book a Table button to move between sections; on a phone, tap the hamburger icon to open the menu
- In Our Menu, choose a category and, if desired, Veg Only or Non-Veg Only; select View Details on any dish to see ingredients and notes
- To add a dish, select Add Dish, complete the form and select Add to Menu (the dish lasts until the page is reloaded)
- To request a table, complete the reservation form with name, e-mail, phone, guests, date and time, then select Confirm Reservation
- Select Make another reservation to return to the form

Step 5: Customise

- Edit the MENU array at the top of js/script.js to change dishes, prices, notes and featured items
- Change colours and fonts through the custom properties at the top of css/style.css
- Replace a dish photograph by overwriting its file in the images folder, or change the image path in the MENU array
- Replace the Unsplash image URLs in index.html with the restaurant's own photographs

14. Glossary of Terms

Term	Definition
HTML5	Current version of the HyperText Markup Language, used to structure the page with semantic elements
CSS3 / Custom Properties	Style language; custom properties (variables such as --gold) store design tokens for reuse
CSS Grid / Flexbox	Layout systems used for card grids and navigation rows
Responsive design	Approach in which one page adapts its layout to different screen sizes
Media query	CSS rule that applies styles only under given conditions, such as a maximum screen width
JavaScript (ES6)	Programming language run by the browser; used for all interactivity
DOM	Document Object Model — the tree of page elements that scripts can read and change
IIFE	Immediately invoked function expression; wraps script.js so that its variables stay private
Event delegation	Handling many elements' events with one listener on a parent element
<dialog> element	Native HTML modal window with built-in focus handling and Escape-to-close
SVG / data URI	Scalable vector graphic; a data URI embeds the image directly in a string, avoiding a file request (used for fallback artwork)
IntersectionObserver	Browser API that reports when elements enter the viewport; used for reveal animations
ARIA	Accessible Rich Internet Applications attributes that expose roles and states to assistive technology
WCAG	Web Content Accessibility Guidelines published by the W3C
Skip link	First link on a page that lets keyboard users jump past the navigation to the main content
Scroll-spy	Technique that highlights the navigation link of the section currently in view
Toast	Small temporary message that confirms an action or reports an error
Client-side validation	Checking user input in the browser before it is accepted
Static website	Site made of fixed files with no server-side processing or database
Veg / Non-veg marker	Green or red square symbol indicating whether a dish is vegetarian

CS23531 – Mini-Project
SDG Mapping

Project Title : SAVOR – MODERN DINING EXPERIENCE: A RESPONSIVE RESTAURANT WEBSITE USING HTML, CSS AND JAVASCRIPT

SDG (No. & Theme)	Activity addressing SDG Theme
SDG 3 – Good Health and Well-being	Every dish carries a vegetarian / non-vegetarian marker, and the site offers Veg Only and Non-Veg Only filters plus an allergen and serving note (dairy, nuts, gluten, shellfish) in the dish details, helping guests make informed dietary choices.
SDG 8 – Decent Work and Economic Growth	Gives a small hospitality business a low-cost online presence with a browsable menu and a table-request form, without paid platforms or backend infrastructure. The menu can be maintained by editing a single data array.
SDG 9 – Industry, Innovation and Infrastructure	Demonstrates lightweight digital adoption: a static, dependency-free front end that runs from plain files, needs no server and keeps its core content, including the dish photographs, usable offline.
SDG 10 – Reduced Inequalities	Accessibility features — skip link, keyboard-operable dialogs, ARIA labels and live regions, visible focus, reduced-motion support and a responsive layout — allow guests with different abilities and devices to use the site.